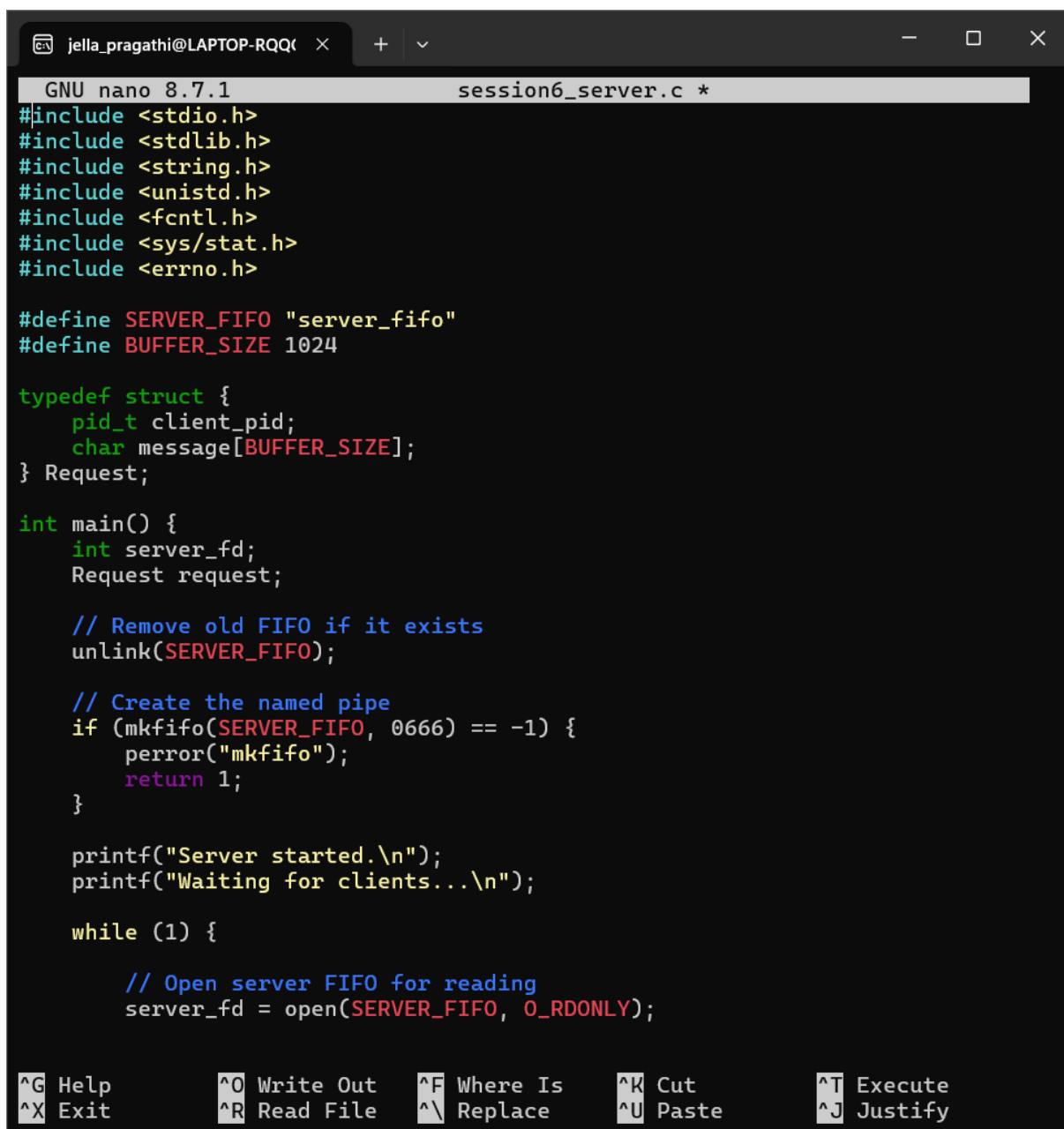


Practical Week 6

Session 6

Task 1

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.



```
GNU nano 8.7.1 session6_server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <errno.h>

#define SERVER_FIFO "server_fifo"
#define BUFFER_SIZE 1024

typedef struct {
    pid_t client_pid;
    char message[BUFFER_SIZE];
} Request;

int main() {
    int server_fd;
    Request request;

    // Remove old FIFO if it exists
    unlink(SERVER_FIFO);

    // Create the named pipe
    if (mkfifo(SERVER_FIFO, 0666) == -1) {
        perror("mkfifo");
        return 1;
    }

    printf("Server started.\n");
    printf("Waiting for clients...\n");

    while (1) {
        // Open server FIFO for reading
        server_fd = open(SERVER_FIFO, O_RDONLY);
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

```
jella_pragathi@LAPTOP-RQ...  GNU nano 8.7.1  session6_server.c *

    if (server_fd == -1) {
        perror("open server_fifo");
        continue;
    }

    // Read client request
    ssize_t bytes = read(server_fd, &request, sizeof(request));

    close(server_fd);

    if (bytes <= 0) {
        continue;
    }

    printf("\nClient PID: %d\n", request.client_pid);
    printf("Message: %s\n", request.message);

    // Create client response FIFO name
    char client_fifo[100];

    snprintf(client_fifo, sizeof(client_fifo),
             "client_%d_fifo", request.client_pid);

    // Send response
    int client_fd = open(client_fifo, O_WRONLY);

    if (client_fd == -1) {
        perror("open client FIFO");
        continue;
    }

    char response[BUFFER_SIZE];

    snprintf(response, sizeof(response),
             "Server received your message: %s",
             request.message);
|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```
request.message);

    write(client_fd, response, strlen(response) + 1);

    close(client_fd);

    printf("Response sent to client %d.\n", request.client_pid);
}

unlink(SERVER_FIFO);

return 0;
|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

GNU nano 8.7.1

session6_client.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "server_fifo"
#define BUFFER_SIZE 1024

typedef struct {
    pid_t client_pid;
    char message[BUFFER_SIZE];
} Request;

int main() {
    Request request;

    // Get client PID
    request.client_pid = getpid();

    // Create unique client FIFO
    char client_fifo[100];

    snprintf(client_fifo, sizeof(client_fifo),
             "client_%d_fifo", request.client_pid);

    unlink(client_fifo);

    if (mkfifo(client_fifo, 0666) == -1) {
        perror("mkfifo");
        return 1;
    }

    // Get message
    printf("Enter message: ");
    fflush(stdout);
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
jella_pragathi@LAPTOP-RQQ( x + v
GNU nano 8.7.1 session6_client.c *
fgets(request.message, BUFFER_SIZE, stdin);

// Remove newline
request.message[strcspn(request.message, "\n")] = '\0';

// Open server FIFO
int server_fd = open(SERVER_FIFO, O_WRONLY);

if (server_fd == -1) {
    perror("open server_fifo");
    unlink(client_fifo);
    return 1;
}

// Send request
write(server_fd, &request, sizeof(request));

close(server_fd);

printf("Message sent to server.\n");

// Wait for response
int client_fd = open(client_fifo, O_RDONLY);

if (client_fd == -1) {
    perror("open client FIFO");
    unlink(client_fifo);
    return 1;
}

char response[BUFFER_SIZE];

ssize_t bytes = read(client_fd, response, sizeof(response) - 1);

if (bytes > 0) {
    response[bytes] = '\0';
    printf("Server response: %s\n", response);
|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

```
printf("Server response: %s\n", response);
}

close(client_fd);

// Remove client FIFO
unlink(client_fifo);

return 0;
}
|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

```
jella_pragathi@LAPTOP-RQQGT555:~$ cd ~/OSSP
./session6_client
Enter message: Hello Server
Message sent to server.
Server response: Server received your message: Hello Server
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ cd ~/OSSP
./session6_client
Enter message: Client 1 message
Message sent to server.
Server response: Server received your message: Client 1 message
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$
```

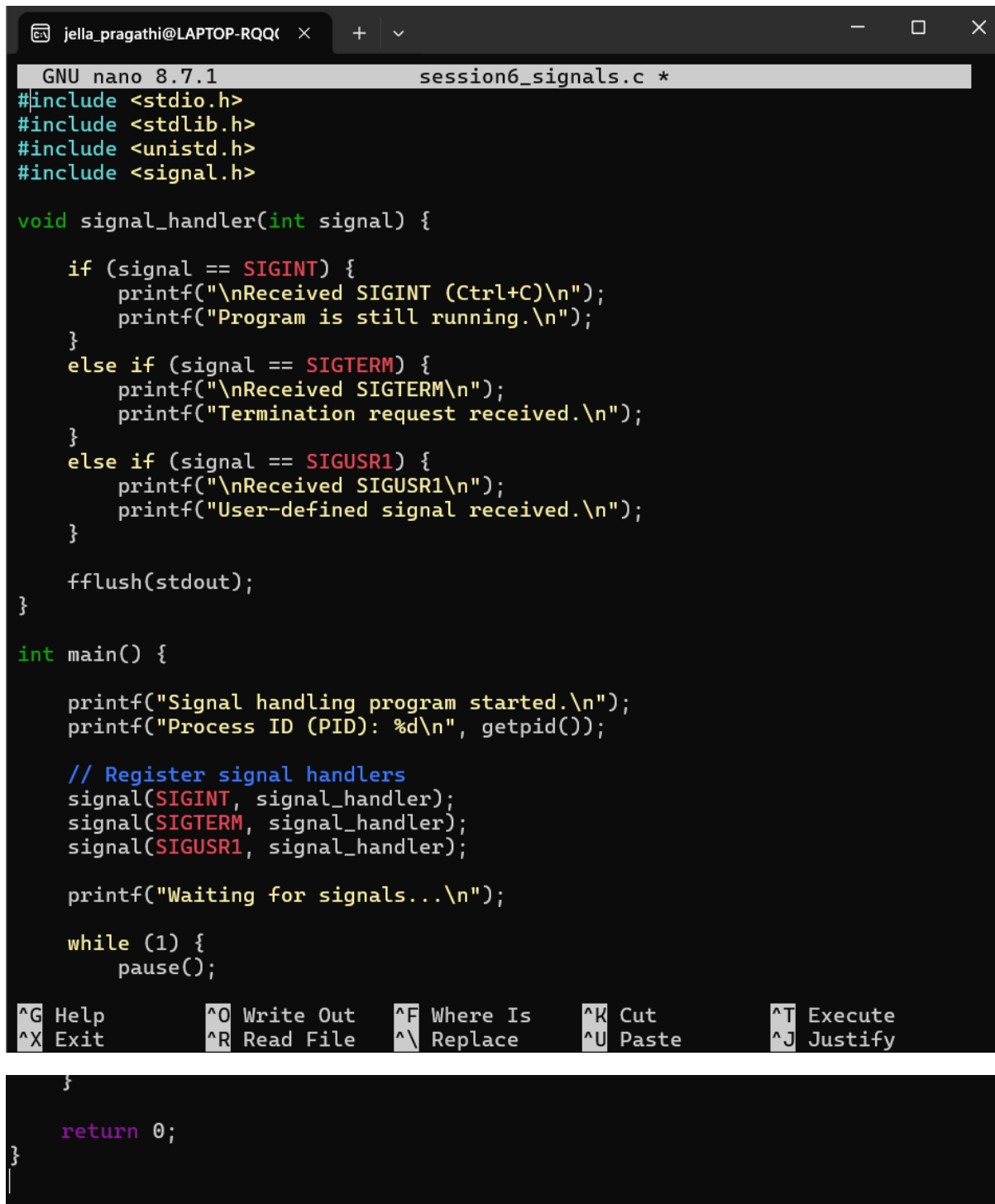
```
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ gcc -Wall -Wextra -g session6_client.c -o session6_client
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ls -l session6_server session6_client
-rwxr-xr-x 1 jella_pragathi jella_pragathi 20328 Sep 17 08:44 session6_client
-rwxr-xr-x 1 jella_pragathi jella_pragathi 18832 Sep 17 08:43 session6_server
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ./session6_server
Server started.
Waiting for clients...

Client PID: 646
Message: Hello Server
Response sent to client 646.

Client PID: 667
Message: Client 1 message
Response sent to client 667.
```

Task 2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.



```
GNU nano 8.7.1 session6_signals.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int signal) {
    if (signal == SIGINT) {
        printf("\nReceived SIGINT (Ctrl+C)\n");
        printf("Program is still running.\n");
    }
    else if (signal == SIGTERM) {
        printf("\nReceived SIGTERM\n");
        printf("Termination request received.\n");
    }
    else if (signal == SIGUSR1) {
        printf("\nReceived SIGUSR1\n");
        printf("User-defined signal received.\n");
    }

    fflush(stdout);
}

int main() {
    printf("Signal handling program started.\n");
    printf("Process ID (PID): %d\n", getpid());

    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Waiting for signals...\n");

    while (1) {
        pause();
    }

    return 0;
}
```

[^]G Help [^]O Write Out [^]F Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify

```
jella_pragathi@LAPTOP-RQQGT555:~$ kill -SIGUSR1 821
jella_pragathi@LAPTOP-RQQGT555:~$ kill -SIGTERM 821
jella_pragathi@LAPTOP-RQQGT555:~$ kill -9 821
jella_pragathi@LAPTOP-RQQGT555:~$ |

jella_pragathi@LAPTOP-RQQGT555:~$ nano session6_signals.c
jella_pragathi@LAPTOP-RQQGT555:~$ gcc -Wall -Wextra -g session6_signals.c -o
session6_signals
jella_pragathi@LAPTOP-RQQGT555:~$ ./session6_signals
Signal handling program started.
Process ID (PID): 821
Waiting for signals...

Received SIGUSR1
User-defined signal received.

Received SIGTERM
Termination request received.
^C
Received SIGINT (Ctrl+C)
Program is still running.
Killed ./session6_signals
jella_pragathi@LAPTOP-RQQGT555:~$ |
```